

Stage 3 Prototype Documentation: UAV Path Visualizer

This document details the design and scope of the evolutionary prototype for the UAV Path Visualizer project.

1. Statement and Justification of Choice

For this project, we chose and have developed a **vertical evolutionary prototype**.

- **Justification For Evolutionary Approach:**

The code developed for this prototype is intended to be the foundation of final application. It is not a throw-away mock up. We have implemented an architecture with emphasis on a SceneManager class which handles the application's state and logic. The code adheres to the quality software development standards, including modular design and clear separation of concerns between UI and backend logic, ensures it can be extended upon in further stages.

- **Justification for vertical approach:**

Our prototype is vertical as it focuses on implementing two core feature completely, from user interface down to back-end data processing (Load 3D scene and Trajectory on UI).

2. Class Definitions

Our prototype is built upon several key classes and logic components.

- 1. class VisualizationSystem:** This is the core class of our whole application. It functions as a primary controller in a Model-View-Controller(MVC) pattern. It does not direct manage data or UI widgets, it orchestrates the flow of exposing a high level API(e.g load scene, load trajectory) to the UI layer. The role is to decouple the UI from backend which allows it to be modified without affecting the other.
- 2. class SceneManager:** Acts as a data model manager for 3D environment. Encapsulates the logic for file I/O and processing of 3D mesh using open3d library. It is responsible for parsing the file formats(.ply,.obj) ,performs data validation and holding geometry in memory as o3d.gemetry.TriangleMesh object. It generates and manages other scene geometries like the ground plae and coordinate axes.
- 3. class ViewerApp:** this class is the 'view' component of the system , It inherits from QMainWindow and is responsible for the entire presentation layer. It

constructs the GUI using PyQt5 widgets, manages layouts and handles the main application event loop. It does not contain any data processing logic. Purpose is to display state of the system to user and capture user input events which will be forwarded to the VisualisationSystem controller

3. Class Member functions:

- **VisualizationSystem**

__init__(config): This is the constructor which initializes the system by instantiating its subordinate components .

load_scene(mesh_path): An API method which handles complex loading operation .(Note given by lecturer)

render_waypoint_view(waypoint_index, callback): This function manages an asynchronous operation. It instantiates a RenderThread worker, connects its render_finished signal to the provided callback slot and starts the thread.

- **SceneManager**

load_mesh(mesh_path): This method handles the technical details of mesh ingestion. It uses pathlib.Path for path manipulation and performs file extension validation.

get_scene_objects(): This function is responsible for data transformation. It converts the internal open3d mesh object into a plotly.graph_objects.Mesh3d trace, translating the numpy arrays of vertices and triangles into the dictionary structure required by the Plotly API.

- **ViewerApp**

_init_ui(): This method constructs the GUI widget tree. It instantiates QWidget containers, applies QVBoxLayout and QHBoxLayout.

_load_mesh_dialog(): A slot that responds to a user event. It utilizes the QFileDialog.getOpenFileName static method to invoke the native OS file selection dialog and filtering for specific file extensions. The selected file path is then passed to the controller.

_update_3d_view(): This method handles the final step of the rendering pipeline. It calls `self.vis_system.renderer.get_html()` to get a serialized HTML representation of the 3D scene and pushes this content to the `QWebEngineView` widget for rendering. This allows local rendering so do not open web browser.

4. Class Inheritance:

ViewerApp(QMainWindow): The `ViewerApp` class derives from `QMainWindow`, the base class for a top-level application window in PyQt. This inheritance provides `ViewerApp` with built-in functionalities, including window management, a predefined layout for menus/toolbars/status bars, and the entire event handling system.

RenderThread(QThread): The `RenderThread` class derives from `QThread` to enable concurrent execution. By this inheriting from `QThread`, an instance of `RenderThread` can be moved to and executed by a separate OS-level thread. We override the `run()` method, which serves as the entry point for the new thread. This allows us to perform the computationally intensive FPV rendering operation without blocking the main GUI thread, thus preventing the application from becoming unresponsive.

5. Scope

The prototype's scope is **vertical**, showing complete, end-to-end implementation of two foundational functionalities. This approach validates the entire software stack, from user interaction to data processing and final rendering.

- **Vertical Slice 1: Scene Loading Data Flow:** This slice proves the primary and first data ingestion and visualisation pipeline. The data flow is:
 1. A clicked signal is emitted by the `QPushButton` in the `ViewerApp`.
 2. The connected slot, `_load_mesh_dialog`, invokes the `VisualizationSystem.load_scene()` method.
 3. The controller assigns to `SceneManager.load_mesh()`, which uses `open3d` to parse the file into a numpy data structure.
 4. The controller then calls `render()`, which triggers the `SceneManager` to convert it's internal mesh into a Plotly trace.
 5. The `PlotlyRenderer` generates an HTML string from this trace.
 6. The `ViewerApp._update_3d_view()` method is called, pushing the HTML to the `QWebEngineView` for final rendering.

Vertical Slice 2: End-to-End Trajectory Data Visualization: This slice shows the system's ability to absorb, process, and render dynamic flight path data onto an existing or non existing 3D scene. This validates the core purpose of the application and the interaction between its data processing and rendering components. The data flow is:

1. A clicked signal is emitted by the "Load Trajectory" QPushButton in the ViewerApp.
2. The connected slot, `_load_trajectory_dialog`, is invoked, which opens a QFileDialog for user input.
3. The selected file path is passed to the controller method, `VisualizationSystem.load_trajectory()`.
4. The controller instantiates a `WaypointParser` and assigns the file I/O and parsing of the JSON data structure. The parser extracts waypoint positions and the flight path order into numpy arrays.
5. A `PathVisualizer` is then instantiated. Its `visualize_trajectory()` method is called to convert the raw waypoint data into a collection of `plotly.graph_objects` traces
6. The controller stores the parser and visualiser instances and calls its main `render()` method.
7. The `render()` method retrieves the static scene objects from the `SceneManager` and the new trajectory objects from the `PathVisualizer`, passing the combined list to the `PlotlyRenderer`.
8. Finally, the `ViewerApp._update_3d_view()` method is called, which gets the complete scene as a serialized HTML string from the `PlotlyRenderer` and pushes it to the `QWebEngineView` for rendering.
Simultaneously, `_update_waypoint_table()` is called to populate the `QTableWidget` with the parsed waypoint data.